

PROJETS Apprentissage automatique supervisé

Modèle de défi de taux de conversion 360 min



Voici le modèle qui illustre les différentes étapes du défi. Dans ce notebook, toutes les étapes d'entraînement et de prédiction sont implémentées pour un modèle très simple (régression logistique à une seule variable). Utilisez ce modèle et n'hésitez pas à modifier les étapes de prétraitement et d'entraînement pour obtenir le modèle avec le meilleur score F1 ! Que la force soit avec vous ! 🍷 🍷

Pour une description détaillée de ce projet, veuillez vous référer à *02-Conversion_rate_challenge.ipynb*.

Bibliothèques d'importation

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, confusion_matrix

import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
import plotly.io as pio
```

```
# setting Jedha color palette as default
pio.templates["jedha"] = go.layout.Template(
    layout_colorway=["#4B9AC7", "#4BE8E0", "#9DD4F3", "#97FBF6", "#2A7FAF", "#
)
pio.templates.default = "jedha"
pio.renderers.default = "svg" # to be replaced by "iframe" if working on JULIE
from IPython.display import display
```

Lire le fichier avec les étiquettes

```
data = pd.read_csv('conversion_data_train.csv')
print('Set with labels (our train+test) :', data.shape)
```

```
Set with labels (our train+test) : (284580, 6)
```

```
data.head()
```

	country	age	new_user	source	total_pages_visited	converted
0	China	22	1	Direct	2	0
1	UK	21	1	Ads	3	0
2	Germany	20	0	Seo	14	1
3	US	23	1	Seo	3	0
4	US	28	1	Direct	3	0

Explorer l'ensemble de données

```
# The dataset is quite big : you must create a sample of the dataset before ma
data_sample = data.sample(10000)
```

Créez votre modèle

Choisissez les variables à utiliser dans le modèle et créez des ensembles d'entraînement et de test.

L'analyse exploratoire des données (EDA) révèle que le nombre total de pages consultées est la variable la plus pertinente. Créons un modèle de base en utilisant initialement cette seule variable : dans les cellules suivantes, nous effectuerons les prétraitements et entraînerons une régression logistique simple (univariée).

```
features_list = ['total_pages_visited']
numeric_indices = [0]
categorical_indices = []
target_variable = 'converted'
```

```
X = data.loc[:, features_list]
Y = data.loc[:, target_variable]

print('Explanatory variables : ', X.columns)
print()
```

```
Explanatory variables :  Index(['total_pages_visited'], dtype='object')
```

```
# Divide dataset Train set & Test set
print("Dividing into train and test sets...")
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.1, random_state=42)
print("...Done.")
print()
```

```
Dividing into train and test sets...
...Done.
```

```
# Put here all the preprocessings
print("Encoding categorical features and standardizing numerical features...")

featureencoder = StandardScaler()
X_train = featureencoder.fit_transform(X_train)
print("...Done")
print(X_train[0:5,:])
```

```
Encoding categorical features and standardizing numerical features...
...Done
[[-0.26070136]
 [ 0.93728655]
 [-0.85969532]
 [-0.56019834]
 [-0.26070136]]
```

```
# Train model
print("Train model...")
classifier = LogisticRegression() #
classifier.fit(X_train, Y_train)
print("...Done.")
```

```
Train model...
...Done.
```

```
# Predictions on training set
print("Predictions on training set...")
Y_train_pred = classifier.predict(X_train)
print("...Done.")
print(Y_train_pred)
print()
```

```
Predictions on training set...
...Done.
[0 0 0 ... 0 0 0]
```

pipeline de test

```
# Use X_test, and the same preprocessings as in training pipeline,  
# but call "transform()" instead of "fit_transform" methods (see example below)  
  
print("Encoding categorical features and standardizing numerical features...")  
  
X_test = featureencoder.transform(X_test)  
print("...Done")  
print(X_test[0:5,:])
```

```
Encoding categorical features and standardizing numerical features...  
...Done  
[[ 0.63778957]  
 [ 0.03879562]  
 [-0.26070136]  
 [-0.26070136]  
 [ 0.63778957]]
```

```
# Predictions on test set  
print("Predictions on test set...")  
Y_test_pred = classifier.predict(X_test)  
print("...Done.")  
print(Y_test_pred)  
print()
```

```
Predictions on test set...  
...Done.  
[0 0 0 ... 0 0 0]
```

Évaluation des performances

```
# WARNING : Use the same score as the one that will be used by Kaggle !  
# Here, the f1-score will be used to assess the performances on the leaderboard  
print("f1-score on train set : ", f1_score(Y_train, Y_train_pred))  
print("f1-score on test set : ", f1_score(Y_test, Y_test_pred))
```

```
f1-score on train set :  0.6938517686692869  
f1-score on test set :  0.7060240963855422
```

```
# You can also check more performance metrics to better understand what your model is doing  
print("Confusion matrix on train set : ")  
print(confusion_matrix(Y_train, Y_train_pred))  
print()  
print("Confusion matrix on test set : ")  
print(confusion_matrix(Y_test, Y_test_pred))  
print()
```

```
Confusion matrix on train set :  
[[246817  1082]  
 [  3280  4943]]
```

```
Confusion matrix on test set :  
[[27384  117]  
 [  371  586]]
```

Notre modèle de base atteint un score F1 de près de 70 %. À présent, n'hésitez pas à affiner votre modèle et à tenter de battre ce score ! 🚀🚀

Entraînez le meilleur classificateur sur toutes les données et utilisez-le pour

faire des prédictions sur X_without_labels.

Avant d'effectuer des prédictions sur le fichier `conversion_data_test.csv`, entraînons notre modèle sur l'ensemble des données contenues dans `conversion_data_train.csv`. Cela permet parfois d'améliorer légèrement le score, car nous utilisons davantage d'exemples pour entraîner le modèle.

```
# Concatenate our train and test set to train your best classifier on all data
X = np.append(X_train,X_test,axis=0)
Y = np.append(Y_train,Y_test)

classifier.fit(X,Y)
```

```
LogisticRegression()
```

```
# Read data without labels
data_without_labels = pd.read_csv('conversion_data_test.csv')
print('Prediction set (without labels) :', data_without_labels.shape)

# Warning : check consistency of features_list (must be the same than the feat
# used by your best classifier)
features_list = ['total_pages_visited']
X_without_labels = data_without_labels.loc[:, features_list]

# Convert pandas DataFrames to arrays before using scikit-learn
print("Convert pandas DataFrames to arrays...")
X_without_labels = X_without_labels.values
print("...Done")

print(X_without_labels[0:5,:])
```

```
Prediction set (without labels) : (31620, 5)
Convert pandas DataFrames to arrays...
...Done
[[16]
```

```
[ 5]
[ 1]
[ 6]
[ 3]]
```

```
# WARNING : PUT HERE THE SAME PREPROCESSING AS FOR YOUR TEST SET
# CHECK YOU ARE USING X_without_labels
print("Encoding categorical features and standardizing numerical features...")

X_without_labels = featureencoder.transform(X_without_labels)
print("...Done")
print(X_without_labels[0:5,:])
```

```
Encoding categorical features and standardizing numerical features...
...Done
[[ 3.33326238]
 [ 0.03879562]
 [-1.15919229]
 [ 0.3382926 ]
 [-0.56019834]]
```

```
# Make predictions and dump to file
# WARNING : MAKE SURE THE FILE IS A CSV WITH ONE COLUMN NAMED 'converted' AND
# WARNING : FILE NAME MUST HAVE FORMAT 'conversion_data_test_predictions_[name]
# where [name] is the name of your team/model separated by a '-'
# For example : [name] = AURELIE-model1
data = {
    'converted': classifier.predict(X_without_labels)
}

Y_predictions = pd.DataFrame(columns=['converted'],data=data)
Y_predictions.to_csv('conversion_data_test_predictions_EXAMPLE.csv', index=False)
```


Analyse des coefficients et interprétation du résultat

Dans ce modèle, nous avons entraîné un modèle avec une seule caractéristique (total_pages_visited), il n'y a donc aucune analyse à effectuer sur l'importance de cette caractéristique 🤔

Une fois que vous aurez intégré davantage de fonctionnalités à votre modèle, prenez le temps d'analyser ses paramètres et d'identifier des leviers d'action pour améliorer le taux de conversion de votre newsletter 😎😎